

Chapter 65: Charts

Scenario 1: Wrapping Chart.js in a standalone component without fighting change detection

Situation: A team is integrating Chart.js into a new standalone Angular application. The first attempt creates the `Chart` instance directly inside `ngOnInit`, reads `@Input()` properties for the dataset, and mutates the chart object imperatively whenever inputs change via `ngOnChanges`. The lead reviewing the PR is worried this will fight Angular's change detection and create subtle bugs as the app migrates to signals.

Question: How would you design a Chart.js wrapper component the "modern Angular" way — standalone, signal-driven, zoneless-friendly — so the chart library and Angular's reactivity model don't step on each other?

Answer: The core idea is to treat the chart instance as a side effect that is *derived from* signal state, not an independent thing kept in sync via lifecycle hooks. Concretely:

1. Accept data via `input()` (signal-based inputs) instead of decorator `@Input()`.
2. Create the `Chart` instance once, in `afterNextRender` (or lazily on first effect run) so it only touches the DOM/canvas after the view is committed — this matters even more once SSR is in play.
3. Use `effect()` to react to signal changes and imperatively update the chart (`chart.data = ...; chart.update()`), rather than destroying/recreating it every time.
4. Because Chart.js does its own internal rendering loop and DOM mutation, run construction and updates inside `NgZone.runOutsideAngular()` so Angular doesn't schedule change detection for every animation frame Chart.js produces internally (tooltips, hover, animations).
5. Clean up with `DestroyRef.onDestroy()` (or `ngOnDestroy` if you prefer classic hooks) calling `chart.destroy()`.

```
import {
  Component, ElementRef, input, effect, viewChild,
  inject, DestroyRef, NgZone, afterNextRender, ChangeDetectionStrategy
} from '@angular/core';
import { Chart, ChartConfiguration, ChartData } from 'chart.js/auto';

@Component({
  selector: 'app-chart',
  standalone: true,
  template: `<canvas #canvasRef></canvas>`,
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class ChartWrapperComponent {
  private readonly canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly zone = inject(NgZone);
  private readonly destroyRef = inject(DestroyRef);

  data = input.required<ChartData<'line'>>();
  options = input<ChartConfiguration<'line'>['options']>({ responsive: true });

  private chart: Chart<'line'> | undefined;
```

```

constructor() {
  afterNextRender(() => {
    this.zone.runOutsideAngular(() => {
      this.chart = new Chart(this.canvasRef().nativeElement, {
        type: 'line',
        data: this.data(),
        options: this.options(),
      });
    });
  });

  // React to signal changes AFTER the chart exists.
  effect(() => {
    const nextData = this.data();
    const nextOptions = this.options();
    if (!this.chart) return; // first run before afterNextRender fires
    this.zone.runOutsideAngular(() => {
      this.chart!.data = nextData;
      this.chart!.options = nextOptions ?? this.chart!.options;
      this.chart!.update();
    });
  });

  this.destroyRef.onDestroy(() => this.chart?.destroy());
}

```

Tradeoffs: Running outside the zone means any code that reads chart callback data (e.g., click handlers that update Angular state) must explicitly re-enter the zone with `this.zone.run(() => ...)` or use signals, which are zone-agnostic and re-render fine either way. This is exactly why signals + `OnPush` + zoneless is a more natural fit for chart-heavy apps than zone-based CD.

Interviewer intent: Tests whether the candidate understands that third-party imperative libraries need to be isolated from Angular's reactivity, and that "signal in, effect updates chart" is the correct one-way data flow — not two-way binding hacks.

Scenario 2: Chart not updating when a signal-based data array changes

Situation: A dashboard chart takes a `data = input<number[]>()`. The parent pushes new points into the array with `array.push(newPoint)` and calls `this.chartData.set(array)`. Visually, nothing happens — the chart stays frozen even though the console shows the array has more elements.

Question: Why doesn't the chart update, and how do you fix it?

Answer: This is a classic reference-equality problem colliding with signals' change detection semantics.

`signal.set(array)` with the *same array reference* (mutated in place via `push`) does not create a new value from `effect()`'s point of view in terms of triggering downstream consumers who compare by identity — but more importantly, even if the `effect()` does re-run (signals always notify on `.set()`, regardless of reference), the chart library itself (e.g., Chart.js) often keeps its own *internal* cached reference to the dataset array and only redraws when you call `chart.update()`. So there are actually two potential bugs stacked:

1. **Signal misuse:** mutating an array in place and calling `.set()` with the *same* reference works for signals (since `set` always emits), but using `.update()` with an in-place push and returning the same array is broken with `computed()` / `OnPush` bindings elsewhere because those often rely on reference change for memoization. Best practice: always produce a new array — `this.chartData.update(arr => [...arr, newPoint])`.
2. **Chart-library staleness:** even after the effect re-runs, if you assign `chart.data.datasets[0].data = newData` but never call `chart.update()`, Chart.js will not repaint. Or if you only mutate the array in place and Chart.js decided to cache a reference to the old array's contents (common performance optimization in charting libs), the redraw might silently no-op.

The fix is both immutable updates on the Angular side, and an explicit imperative `update()` call inside the effect:

```
effect(() => {
  const points = this.data(); // new array each time
  if (!this.chart) return;
  this.chart.data.labels = points.map((_, i) => i);
  this.chart.data.datasets[0].data = points; // reassign, don't mutate in place
  this.chart.update('none'); // 'none' skips animation for streaming updates
});
```

And upstream:

```
addPoint(value: number) {
  this.chartData.update(current => [...current, value]); // new reference
}
```

Interviewer intent: Probes understanding of the difference between Angular's own reactivity (which mostly doesn't care about reference equality for `effect` / `signal` re-emission) versus third-party library internals that DO require explicit "commit" calls, plus general immutability discipline.

Scenario 3: Memory leak from an undisposed chart instance in a routed dashboard

Situation: Users report that after navigating between five or six different dashboard pages (each containing 2-3 charts), the browser tab's memory usage keeps climbing and the app eventually becomes sluggish. Profiling with Chrome DevTools' heap snapshot shows detached canvas elements and retained `Chart` instances that should have been garbage collected.

Question: Walk through how you'd diagnose this and what the fix looks like.

Answer: Diagnosis steps:

1. Take two heap snapshots (before and after navigating away from a chart-heavy route several times), then use the "Comparison" view in DevTools to look for growing counts of `Chart`, `CanvasRenderingContext2D`, or detached `HTMLCanvasElement` nodes.
2. Detached DOM nodes retained in memory almost always mean something is still holding a JS reference to them — typically an event listener registered by the chart library (resize listeners, animation frame callbacks) or a subscription that was never torn down.

3. Common root causes: (a) the component's `ngOnDestroy / DestroyRef` cleanup never calls `chart.destroy()`; (b) a `ResizeObserver` or `window.addEventListener('resize', ...)` was added manually and never disconnected; (c) an RxJS subscription to a streaming data source (WebSocket) feeding the chart isn't unsubscribed, so the callback keeps firing and keeps a closure reference to the destroyed component instance alive.

Fix — ensure every side effect registered by the wrapper is torn down deterministically using `DestroyRef`:

```
import { Component, ElementRef, ViewChild, inject, DestroyRef, afterNextRender } from
 '@angular/core';
import { Chart } from 'chart.js/auto';

@Component({
  selector: 'app-leak-safe-chart',
  standalone: true,
  template: `<canvas #canvasRef></canvas>`,
})
export class LeakSafeChartComponent {
  private readonly canvasRef = ViewChild.required<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly destroyRef = inject(DestroyRef);
  private chart?: Chart;
  private resizeObserver?: ResizeObserver;

  constructor() {
    afterNextRender(() => {
      const el = this.canvasRef().nativeElement;
      this.chart = new Chart(el, { type: 'bar', data: { labels: [], datasets: [] } });

      this.resizeObserver = new ResizeObserver(() => this.chart?.resize());
      this.resizeObserver.observe(el.parentElement!);

      // Register cleanup right where the resource was created – no risk of forgetting it.
      this.destroyRef.onDestroy(() => {
        this.resizeObserver?.disconnect();
        this.chart?.destroy();
        this.chart = undefined;
      });
    });
  }
}
```

Key principle: **register the teardown in the same place you register the resource**. Don't rely on a single giant `ngOnDestroy` at the bottom of the class that people forget to update when they add a new subscription elsewhere.

Interviewer intent: Tests real-world debugging methodology (heap snapshots, detached nodes) plus disciplined resource-cleanup habits, not just "call `destroy()`" as a rote answer.

Scenario 4: Real-time streaming chart with high-frequency WebSocket updates

Situation: A monitoring dashboard receives ~50 data points per second over a WebSocket for a live sensor feed. The naive implementation calls `chart.update()` on every message. The UI becomes janky, CPU usage spikes, and eventually the browser tab warns about high memory usage.

Question: How do you architect this so it stays smooth?

Answer: Several complementary techniques:

1. **Batch updates with `requestAnimationFrame` or `bufferTime`.** Don't redraw once per message; coalesce many messages that arrive within one frame into a single chart update.
2. **Throttle to the display's refresh rate**, not the data rate — no chart needs 50 updates/sec because the human eye and the monitor can't perceive more than ~60fps, and most dashboards look fine at 4-10 updates/sec.
3. **Cap the dataset size (sliding window).** Never let the in-memory array grow unbounded — drop old points once you exceed e.g. 500 points, both to bound memory and to keep Chart.js's redraw cost constant.
4. **Disable animation** (`update('none')`) for streaming charts — animating every frame is pure wasted CPU for data that's already stale by the time the animation finishes.
5. **Run outside Angular's zone** so each WebSocket message doesn't trigger a full change-detection pass.

```
import { Component, DestroyRef, inject, signal, effect, NgZone, viewChild, ElementRef,
afterNextRender } from '@angular/core';
import { Subject } from 'rxjs';
import { bufferTime, filter } from 'rxjs/operators';
import { Chart } from 'chart.js/auto';

@Component({
  selector: 'app-live-chart',
  standalone: true,
  template: `<canvas #canvasRef></canvas>`,
})
export class LiveChartComponent {
  private readonly canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>(
    'canvasRef'
  );
  private readonly zone = inject(NgZone);
  private readonly destroyRef = inject(DestroyRef);
  private readonly incoming$ = new Subject<number>();
  private chart?: Chart<'line'>;
  private readonly MAX_POINTS = 500;
  private readonly buffer: number[] = [];

  constructor() {
    afterNextRender(() => {
      this.zone.runOutsideAngular(() => {
        this.chart = new Chart(this.canvasRef().nativeElement, {
          type: 'line',
          data: { labels: [], datasets: [{ data: [], data: [] as number[] }] },
        });
      });
    });
  }
}
```

```

        options: { animation: false, responsive: true, scales: { x: { display: false } }
    },
    });

    const sub = this.incoming$
        .pipe(bufferTime(200), filter(batch => batch.length > 0))
        .subscribe(batch => this.applyBatch(batch));

    this.destroyRef.onDestroy(() => {
        sub.unsubscribe();
        this.chart?.destroy();
    });
    });
    });
}

/** Called by the websocket message handler. */
push(value: number) {
    this.incoming$.next(value);
}

private applyBatch(batch: number[]) {
    const ds = this.chart!.data.datasets[0].data as number[];
    ds.push(...batch);
    if (ds.length > this.MAX_POINTS) {
        ds.splice(0, ds.length - this.MAX_POINTS);
    }
    this.chart!.update('none');
}
}

```

Tradeoffs: `bufferTime(200)` trades a small amount of latency (up to 200ms) for a ~10x reduction in redraw frequency — usually an easy call for monitoring dashboards where humans don't need sub-100ms precision. If true low-latency is required (e.g., trading terminals), switch to a canvas-native or WebGL renderer (e.g., `uPlot`, `lightweight-charts`) instead of `Chart.js`, which is not built for tens of updates/sec at scale.

Interviewer intent: Assesses whether the candidate thinks about performance holistically — batching, windowing, animation, and zone isolation together — rather than reaching for a single silver-bullet fix.

Scenario 5: Responsive resizing with `ResizeObserver` instead of window resize events

Situation: A chart is embedded inside a resizable split-pane layout. The existing code listens to `window:resize` to call `chart.resize()`, but the chart doesn't update at all when the user drags the pane divider (the window itself never resizes, just the container div).

Question: How do you make the chart responsive to *container* resizing rather than window resizing, and how do you wire that up cleanly in Angular?

Answer: `window:resize` only fires when the browser viewport changes size — it's blind to layout changes caused by flexbox/grid reflows, sidebar toggles, or split panes. The correct primitive is `ResizeObserver`, which observes a specific element's box size and fires whenever *that element* changes size for any reason.

Implementation notes:

- Observe the canvas's **parent container**, not the canvas itself — many chart libraries set the canvas's own size, so observing the canvas can create a resize-observe-loop (canvas resizes → observer fires → library resizes canvas again → ...).
- Debounce/throttle the callback slightly (charts libraries' own `resize()` is often already efficient, but doing it on every intermediate frame during a drag is wasteful).
- Disconnect in `DestroyRef.onDestroy()`.

```
import { Component, ElementRef, ViewChild, inject, DestroyRef, afterNextRender, NgZone }
from '@angular/core';
import { Chart } from 'chart.js/auto';

@Component({
  selector: 'app-responsive-chart',
  standalone: true,
  template: `
    <div #containerRef class="chart-container">
      <canvas #canvasRef></canvas>
    </div>
  `,
  styles: [`.chart-container { width: 100%; height: 100%; min-height: 200px; }`],
})
export class ResponsiveChartComponent {
  private readonly containerRef = ViewChild<ElementRef<HTMLDivElement>>
('containerRef');
  private readonly canvasRef = ViewChild<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly zone = inject(NgZone);
  private readonly destroyRef = inject(DestroyRef);
  private chart?: Chart;
  private resizeTimer?: ReturnType<typeof setTimeout>;

  constructor() {
    afterNextRender(() => {
      this.zone.runOutsideAngular(() => {
        this.chart = new Chart(this.canvasRef().nativeElement, {
          type: 'bar',
          data: { labels: [], datasets: [] },
          options: { responsive: true, maintainAspectRatio: false },
        });

        const observer = new ResizeObserver(() => {
          clearTimeout(this.resizeTimer);
          this.resizeTimer = setTimeout(() => this.chart?.resize(), 50);
        });
        observer.observe(this.containerRef().nativeElement);

        this.destroyRef.onDestroy(() => {
          clearTimeout(this.resizeTimer);
          observer.disconnect();
          this.chart?.destroy();
        });
      });
    });
  }
}
```

```

    });
  });
});
}
}

```

Also set `maintainAspectRatio: false` so the chart truly fills the flexible container rather than being constrained by an aspect-ratio calculation based on stale dimensions.

Interviewer intent: Checks whether the candidate knows the practical difference between viewport-level and element-level resize detection, and can avoid the classic `ResizeObserver` feedback-loop bug.

Scenario 6: Exporting a chart as a PNG and as a PDF

Situation: Product wants an "Export" button on every chart: one option downloads a PNG snapshot, another generates a PDF report containing several charts plus a title page.

Question: How would you implement both export paths, and what pitfalls exist for canvas-based chart libraries?

Answer:

- **PNG export** is straightforward because `<canvas>` exposes `toDataURL()` / `toBlob()` directly. Chart.js also exposes `chart.toBase64Image()` as a convenience wrapper. The gotcha: if `preserveDrawingBuffer` isn't relevant for 2D canvas (that's a WebGL concern), but for **WebGL-based charts** (e.g., some ECharts renderers, three.js overlays) you must set `preserveDrawingBuffer: true` at context-creation time, or the buffer will be cleared before you can read it, producing a blank export.
- **PDF export** typically composes multiple canvases into a single document using a library like `jsPDF`, embedding each chart's PNG data URL as an image on a page. Since `jsPDF` and `canvas-to-image` work in raster form, keep in mind resolution: export at 2x-3x device pixel ratio for crisp print output, not the on-screen render size.
- Because both operations need direct canvas access, keep a reference to the underlying canvas element (or the chart instance) available to the export logic, and guard against calling export before `afterNextRender` has actually created the chart.
- Wrap the actual `toDataURL` / PDF generation in `NgZone.runOutsideAngular` since it can be CPU-heavy for multi-chart PDFs — you don't want it to block the zone's tracked change-detection cycle (relevant when zone-based).

```

import { Component, ElementRef, ViewChild, inject, NgZone } from '@angular/core';
import { Chart } from 'chart.js/auto';
import jsPDF from 'jspdf';

@Component({
  selector: 'app-exportable-chart',
  standalone: true,
  template: `
    <canvas #canvasRef></canvas>
    <button (click)="exportPng()">Export PNG</button>
    <button (click)="exportPdf()">Export PDF</button>
  `,
})

```

```

export class ExportableChartComponent {
  private readonly canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly zone = inject(NgZone);
  chart?: Chart;

  exportPng() {
    if (!this.chart) return;
    const link = document.createElement('a');
    link.href = this.chart.toBase64Image('image/png', 1.0);
    link.download = 'chart.png';
    link.click();
  }

  async exportPdf() {
    if (!this.chart) return;
    await this.zone.runOutsideAngular(async () => {
      const imgData = this.chart!.toBase64Image('image/png', 1.0);
      const pdf = new jsPDF({ orientation: 'landscape', unit: 'pt' });
      const canvas = this.canvasRef().nativeElement;
      const ratio = canvas.height / canvas.width;
      const width = 700;
      pdf.text('Monthly Report', 40, 40);
      pdf.addImage(imgData, 'PNG', 40, 60, width, width * ratio);
      pdf.save('report.pdf');
    });
  }
}

```

Tradeoffs: For multi-chart PDF reports, generating everything client-side keeps the backend simple but can be slow/memory-heavy for large reports; a server-side rendering pipeline (headless Chromium + Puppeteer) scales better for reports with dozens of charts but adds infrastructure complexity.

Interviewer intent: Tests awareness that canvas export has real gotchas (WebGL buffer preservation, DPI/resolution for print) beyond just calling a library function.

Scenario 7: Chart accessibility for screen reader users

Situation: An accessibility audit flags that the dashboard's charts are completely invisible to screen reader users — the `<canvas>` element exposes no semantic content, so a blind user has no way to know what data is being shown.

Question: How do you make a canvas-based chart accessible without abandoning the visual chart library?

Answer: A `<canvas>` is a single opaque bitmap to assistive technology, so accessibility must be layered on top rather than expected "for free." A solid approach combines several techniques:

1. `role="img"` plus a meaningful `aria-label` summarizing the chart's takeaway (not just "chart" — e.g., "Line chart showing monthly revenue rising from \$10k in January to \$45k in June").
2. A visually-hidden data table (`class="sr-only"` / `cdk-visually-hidden`) that duplicates the chart's underlying data in a real `<table>` — screen readers navigate tables far better than they can interpret a canvas description.

3. **Live region announcements** for real-time/streaming charts, using `aria-live="polite"`, so significant changes (e.g., threshold breaches) get announced without spamming every single data point.
4. **Keyboard navigation** for interactive charts (tooltips on hover need an equivalent on focus) — e.g., render invisible focusable elements over each data point, or provide a "data table view" toggle button that fully replaces the chart for keyboard/AT users.
5. Respect `prefers-reduced-motion` to disable chart animations for users who've indicated motion sensitivity.

```
import { Component, computed, input } from '@angular/core';

@Component({
  selector: 'app-accessible-chart',
  standalone: true,
  template: `
    <div class="chart-wrapper" role="img" [attr.aria-label]="summary()">
      <canvas #canvasRef aria-hidden="true"></canvas>
    </div>

    <table class="sr-only">
      <caption>{{ title() }} - data table</caption>
      <thead>
        <tr><th scope="col">Month</th><th scope="col">Revenue (USD)</th></tr>
      </thead>
      <tbody>
        @for (row of rows(); track row.month) {
          <tr><td>{{ row.month }}</td><td>{{ row.value | currency }}</td></tr>
        }
      </tbody>
    </table>
  `,
  styles: [`.sr-only {
    position: absolute; width: 1px; height: 1px;
    padding: 0; margin: -1px; overflow: hidden;
    clip: rect(0,0,0,0); white-space: nowrap; border: 0;
  }`],
})
export class AccessibleChartComponent {
  title = input.required<string>();
  rows = input.required<{ month: string; value: number }[]>();

  summary = computed(() => {
    const data = this.rows();
    if (!data.length) return `{{this.title()}}: no data available.`;
    const first = data[0], last = data[data.length - 1];
    const trend = last.value >= first.value ? 'rising' : 'falling';
    return `{{this.title()}}: ${trend} from ${first.value} in ${first.month} to ${last.value} in ${last.month}.`;
  });
}
```

Tradeoffs: A hidden data table adds DOM weight and duplicate markup to maintain, but it's currently the most reliable pattern across screen readers (canvas + ARIA alone is poorly supported). Some teams instead offer an explicit "View as table" toggle button rather than a permanently visually-hidden table, which also helps low-vision/cognitive-accessibility users, not just screen reader users.

Interviewer intent: Tests whether the candidate treats accessibility as a first-class design constraint for data visualization, not an afterthought bolted onto a purely visual widget.

Scenario 8: SSR crashes because the chart library assumes `window/document/canvas` exist

Situation: The team enables Angular Universal (SSR) for SEO reasons. On the server, the build immediately throws `ReferenceError: window is not defined` or `HTMLCanvasElement is not defined` coming from deep inside the chart library's initialization code, even though the chart component isn't supposed to render meaningful content server-side anyway.

Question: How do you prevent third-party charting libraries from crashing SSR, while still rendering real charts on the client?

Answer: The root cause: many charting libraries (Chart.js, D3 with certain plugins, ECharts) reference browser globals (`window`, `document`, `navigator`, `HTMLCanvasElement`) at **module import time** or in constructor logic, and Node has none of these unless polyfilled. There are three complementary defenses:

1. **Guard construction with** `isPlatformBrowser(inject(PLATFORM_ID))` so the `new Chart(...)` call — and ideally the *import* of the chart library — never executes on the server.
2. **Defer the import itself**, using a dynamic `import()` inside the browser-only branch, so the module's top-level code (which may reference `window`) never even loads during server-side bundling/execution.
3. **Prefer** `afterNextRender`, which Angular guarantees runs only in the browser, after the first render — this is actually the cleanest idiom in modern Angular and makes the `isPlatformBrowser` check largely redundant for this specific case, though keeping both is a defensive habit some teams prefer.
4. Render a lightweight, SSR-safe placeholder (skeleton, `<noscript>` fallback, or a server-rendered static image/description) so users see *something* meaningful before hydration/client JS finishes loading the real chart — this also helps SEO show meaningful text content instead of an empty canvas.

```
import {
  Component, ElementRef, ViewChild, inject, DestroyRef,
  afterNextRender, PLATFORM_ID
} from '@angular/core';
import { isPlatformBrowser } from '@angular/common';

@Component({
  selector: 'app-ssr-safe-chart',
  standalone: true,
  template: `
    @if (!ready()) {
      <div class="chart-skeleton" aria-hidden="true">Loading chart...</div>
    }
    <canvas #canvasRef [class.hidden]="!ready()"></canvas>
  `,
})
```

```

export class SsrSafeChartComponent {
  private readonly canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly platformId = inject(PLATFORM_ID);
  private readonly destroyRef = inject(DestroyRef);
  ready = signal(false);

  constructor() {
    // afterNextRender's default phase only runs in the browser, after the
    // DOM has been committed – but we keep isPlatformBrowser too for
    // clarity/defense-in-depth for teams unfamiliar with the guarantee.
    afterNextRender(async () => {
      if (!isPlatformBrowser(this.platformId)) return;

      // Dynamic import prevents the module's top-level `window` references
      // from ever being evaluated during server-side bundling/execution.
      const { Chart } = await import('chart.js/auto');
      const chart = new Chart(this.canvasRef().nativeElement, {
        type: 'bar',
        data: { labels: [], datasets: [] },
      });
      this.ready.set(true);
      this.destroyRef.onDestroy(() => chart.destroy());
    });
  }
}

```

Also worth checking: some meta-frameworks/build tools let you mark a package as `ssr: { noExternal / external }` or exclude it from server bundling entirely in `angular.json / vite` config, which sidesteps needing dynamic import in every consuming component.

Tradeoffs: Skeletons improve perceived performance but add a layout-shift risk if their size doesn't match the eventual chart — always reserve final height/width via CSS to avoid CLS penalties.

Interviewer intent: Tests whether the candidate understands *why* SSR breaks (module-load-time global access vs. runtime access) and knows Angular's specific `afterNextRender / isPlatformBrowser` tools for solving it, rather than just saying "check if window exists" without knowing where the failure actually originates.

Scenario 9: Chart flickers/re-creates on every OnPush parent re-render

Situation: A chart component receives its dataset as an `@Input() data: ChartPoint[]` object literal built inline in the parent template: `<app-chart [data]="{points: model.points, color: 'blue'}">`. Every time the parent's unrelated state changes (e.g., a sidebar toggle), the chart visibly flickers and rebuilds from scratch, even though the actual chart data hasn't changed.

Question: What's causing the unnecessary rebuild, and how do you fix it using Angular's modern primitives?

Answer: The inline object literal `{points: ..., color: 'blue'}` is a **new object reference on every parent change-detection pass**, regardless of whether its contents changed. If the child's `effect() / ngOnChanges` treats "input reference changed" as "data changed, tear down and rebuild the chart," it will rebuild unnecessarily on literally any parent re-render.

Fixes, layered:

1. **Stop constructing object literals in templates.** Compute the object once in the parent's TS with a `computed()` signal so it's only a new reference when its actual dependencies change.
2. **In the child, diff before acting.** Inside the `effect()`, compare relevant fields (or use a cheap hash/`JSON.stringify` for small payloads, or better, deep-equal only the fields the chart cares about) before calling `chart.destroy()` /recreate — prefer *updating* the existing chart's data (`chart.data = ...; chart.update()`) over destroying and recreating it, since update is far cheaper and avoids the flicker entirely.
3. **Use `OnPush`** on both parent and child (should already be default in modern standalone components) so unrelated state changes don't even trigger the child's CD pass to begin with — though note this doesn't fully solve it alone if the input binding itself still produces a new reference each time input creation code runs.

```
// Parent – memoize with computed() instead of building an object in the template.
@Component({ /* ... */ })
export class DashboardComponent {
  private readonly rawPoints = signal<ChartPoint[]>([]);
  private readonly color = signal('blue');

  // Only changes reference when rawPoints or color actually change.
  chartConfig = computed(() => ({ points: this.rawPoints(), color: this.color() }));
}
```

```
<app-chart [data]="chartConfig()" />
```

```
// Child – update in place rather than destroy/recreate.
effect(() => {
  const cfg = this.data();
  if (!this.chart) return;
  this.chart.data.datasets[0].data = cfg.points;
  this.chart.data.datasets[0].borderColor = cfg.color;
  this.chart.update();
});
```

Tradeoffs: `computed()` memoization is nearly free and should be the default habit; deep-equality checks inside the child add a small CPU cost per update but protect the component even when a careless consumer passes fresh object literals — a good defensive layer for a reusable/shared chart component published as a library.

Interviewer intent: Verifies the candidate understands reference-vs-value semantics in Angular templates and connects it to a very common, very real perf bug — literal objects/arrays created inline in templates causing cascading unnecessary work downstream.

Scenario 10: Rendering hundreds of small sparkline charts in a table without freezing the UI

Situation: A financial table lists 400 rows, each with a tiny sparkline chart showing a 30-day price trend. Rendering all 400 charts on initial load takes several seconds and freezes the main thread; scrolling is janky.

Question: How would you make this scale?

Answer: The key insight is that most of those 400 charts are **off-screen** at any given time, so the first optimization is: don't construct chart instances for rows the user can't see.

1. **Virtual scrolling** via Angular CDK's `cdk-virtual-scroll-viewport` so only the ~20 visible rows are ever in the DOM, meaning only ~20 chart instances exist at once — this alone often solves 90% of the problem.
2. **Defer chart construction with `@defer` (viewport trigger)** for any charts that are individually below-the-fold within a non-virtualized layout, so heavy chart libraries only initialize when they scroll into view.
3. **Use a lighter-weight rendering technique for sparklines specifically** — a full Chart.js instance (with its event listeners, plugins, animation engine) is overkill for a 40x16px sparkline. Hand-rolled SVG `<path>` generation from the data array, or a minimal canvas draw routine without a full charting library, is dramatically cheaper per-instance at this scale.
4. **Batch construction across a frame budget** using `requestIdleCallback` (or chunked `setTimeout`) if you must construct many charts eagerly, so you don't block the main thread in one long synchronous burst.

```
import { Component, input, computed } from '@angular/core';

// Lightweight SVG sparkline – no chart library overhead per row.
@Component({
  selector: 'app-sparkline',
  standalone: true,
  template: `
    <svg [attr.viewBox]="viewBox()" preserveAspectRatio="none" class="sparkline">
      <path [attr.d]="pathData()" fill="none" stroke="currentColor" stroke-width="1.5" />
    </svg>
  `,
  styles: [`.sparkline { width: 60px; height: 20px; }`],
})
export class SparklineComponent {
  values = input.required<number[]>();
  width = 60;
  height = 20;

  viewBox = computed(() => `0 0 ${this.width} ${this.height}`);

  pathData = computed(() => {
    const vals = this.values();
    if (!vals.length) return '';
    const min = Math.min(...vals);
    const max = Math.max(...vals);
    const range = max - min || 1;
    const stepX = this.width / (vals.length - 1 || 1);
```

```

return vals
  .map((v, i) => {
    const x = i * stepX;
    const y = this.height - ((v - min) / range) * this.height;
    return `${i === 0 ? 'M' : 'L'}${x.toFixed(1)},${y.toFixed(1)}`;
  })
  .join(' ');
});
}

```

```

<cdk-virtual-scroll-viewport itemSize="40" class="table-viewport">
  <div *cdkVirtualFor="let row of rows" class="row">
    {{ row.symbol }}
    <app-sparkline [values]="row.last30Days" />
  </div>
</cdk-virtual-scroll-viewport>

```

Tradeoffs: Hand-rolled SVG sparklines lose the rich interaction (tooltips, zoom) a full chart library provides, but that's the correct tradeoff at 40x16px scale where none of those interactions are usable anyway. Virtual scrolling adds complexity (fixed/estimated row heights, scroll-position restoration) but is close to mandatory once row counts get into the hundreds.

Interviewer intent: Tests whether the candidate can reason about "right tool for the job" at scale — recognizing when a heavyweight charting library is the wrong choice entirely, not just optimizing its usage.

Scenario 11: Coordinating multiple linked charts (crosshair/tooltip sync) without tight coupling

Situation: A dashboard shows three charts stacked vertically, all sharing the same time axis. Hovering over any one chart should show a synchronized crosshair and tooltip at the same X position across all three charts. The charts are otherwise independent, reusable components.

Question: How do you implement cross-chart synchronization while keeping each chart component decoupled and reusable?

Answer: This is a shared-state broadcasting problem, best solved with a small injectable state/service scoped to the dashboard, rather than direct chart-to-chart references (which would break reusability and testability).

Design:

1. Create a `ChartSyncService`, `providedIn` at the dashboard component level (not root) so each dashboard instance gets an isolated sync state — using `signal<number | null>()` for the currently hovered X-index/timestamp.
2. Each chart component, on hover (mousemove over canvas), computes the nearest data index and calls `syncService.setActiveIndex(index)`.
3. Each chart component also has an `effect()` that reads `syncService.activeIndex()` and draws its own crosshair overlay at that index — including on charts the user isn't currently hovering over.
4. On `mouseleave`, clear the shared index.

```
import { Injectable, signal } from '@angular/core';

@Injectable() // provided at the dashboard component, not root – one instance per dashboard
export class ChartSyncService {
  readonly activeIndex = signal<number | null>(null);
  setActiveIndex(index: number | null) { this.activeIndex.set(index); }
}
```

```
@Component({
  selector: 'app-linked-chart',
  standalone: true,
  template: `<canvas #canvasRef (mousemove)="onHover($event)" (mouseleave)="onLeave()">
</canvas>`,
})
export class LinkedChartComponent {
  private readonly sync = inject(ChartSyncService);
  private readonly canvasRef = viewChild.required<ElementRef<HTMLCanvasElement>>
('canvasRef');
  data = input.required<number[]>();
  private chart?: Chart<'line'>;

  constructor() {
    effect(() => {
      const idx = this.sync.activeIndex();
      this.drawCrosshair(idx); // no-ops harmlessly if idx is null or out of range
    });
  }

  onHover(evt: MouseEvent) {
    const idx = this.pixelToIndex(evt.offsetX);
    this.sync.setActiveIndex(idx);
  }

  onLeave() { this.sync.setActiveIndex(null); }

  private pixelToIndex(x: number): number { /* map pixel position to data index */ return 0;
}
  private drawCrosshair(index: number | null) { /* re-render chart with overlay
plugin/annotation at index */ }
}
```

```
<!-- dashboard.component.html -->
<div [providers]="..."> <!-- conceptually: provide ChartSyncService at this component -->
  <app-linked-chart [data]="series1()" />
  <app-linked-chart [data]="series2()" />
  <app-linked-chart [data]="series3()" />
</div>
```

```
@Component({
  selector: 'app-dashboard',
  standalone: true,
  providers: [ChartSyncService], // scoped instance shared by all children via DI
  imports: [LinkedChartComponent],
  template: `...`,
})
export class DashboardComponent {}
```

Tradeoffs: A shared service is simpler and more idiomatic in Angular than an event bus or manual `@Output/@Input` chaining across three siblings (which would require lifting state to a common ancestor and threading callbacks). The service approach does mean charts are no longer usable 100% standalone outside a `ChartSyncService`-providing context, but that's an acceptable and explicit contract, easily satisfied with an optional injection (`inject(ChartSyncService, { optional: true })`) for charts used outside any dashboard.

Interviewer intent: Tests whether the candidate reaches for Angular DI scoping (component-level providers) to solve sibling-communication problems elegantly, instead of over-engineering with NgRx or prop-drilling.

Scenario 12: D3 chart taking over the DOM and conflicting with Angular's rendering

Situation: A developer builds a D3.js chart that uses `d3.select(element).selectAll('rect').data(data).join('rect')...` to manage SVG elements directly. Occasionally, after an Angular re-render, some D3-managed elements vanish or duplicate unpredictably. The bug is intermittent and hard to reproduce.

Question: What's the underlying conflict, and how do you resolve it?

Answer: D3 and Angular are both trying to own and mutate the same DOM subtree using two completely different reconciliation models: D3 uses its own imperative `.data().join()` binding to decide what to add/update/remove, while Angular's template engine (even with fine-grained signals) assumes it fully owns and controls the DOM nodes produced by its own template bindings. If Angular's template contains a `<svg>/<g>` with `*ngFor/@for` structural directives generating child elements, and D3 *also* tries to manage children of that same node, they will corrupt each other's bookkeeping — Angular might remove/reorder nodes D3 doesn't know about, and D3's `.join()` will get confused about which DOM nodes still "belong" to which data items.

The fix is architectural: **give D3 (or any DOM-manipulating library) sole, exclusive ownership of one specific host element, and never let Angular's template put structural directives inside that subtree.**

```
@Component({
  selector: 'app-d3-chart',
  standalone: true,
  // The svg is a single static element in Angular's template – Angular never
  // touches anything *inside* it. Everything inside is D3's exclusive domain.
  template: `<svg #svgRef></svg>`,
})
export class D3ChartComponent {
  private readonly svgRef = viewChild.required<ElementRef<SVGSVGElement>>('svgRef');
  data = input.required<{ label: string; value: number }[]>();
```

```

private readonly destroyRef = inject(DestroyRef);

constructor() {
  afterNextRender(() => {
    const svg = d3.select(this.svgRef().nativeElement);

    effect(() => {
      const rows = this.data();
      // D3 fully owns reconciliation of these <rect> elements – Angular's
      // change detector never walks into this subtree because there is no
      // Angular template syntax (*for, bindings) describing these nodes.
      svg.selectAll<SVGRectElement, typeof rows[0]>('rect')
        .data(rows, d => d.label)
        .join(
          enter => enter.append('rect').attr('height', 0),
          update => update,
          exit => exit.remove()
        )
        .attr('width', 20)
        .attr('height', d => d.value)
        .attr('x', (_, i) => i * 25);
    });

    this.destroyRef.onDestroy(() => svg.selectAll('*').remove());
  });
}

```

Key rule: **never mix Angular structural directives** (`*ngFor`, `@for`, **conditional rendering**) **with imperative DOM libraries inside the same container**. If you need Angular-rendered elements (say, HTML tooltips) positioned relative to D3 elements, render them in a *sibling* container using Angular's own template engine, and compute positions imperatively (e.g., via a signal updated from D3's tick/zoom callbacks) rather than nesting one inside the other's managed subtree.

Interviewer intent: Tests deep understanding of DOM-ownership boundaries — a subtle but critical concept whenever integrating any imperative rendering library (D3, Three.js, video players) with a declarative framework.

Scenario 13: `effect()` running in an infinite loop when updating chart state

Situation: A developer writes an `effect()` that reads a `zoomLevel` signal to redraw a chart, but inside that same effect they also call `this.zoomLevel.set(clampedValue)` to enforce min/max bounds. The app immediately hits NG0600: `writing to signals is not allowed in a computation` or, if using `allowSignalWrites`, spins in an infinite effect loop, pegging the CPU.

Question: Why does this happen, and what's the correct pattern for "read a signal, derive/clamp a value, and write it back"?

Answer: `effect()` automatically tracks every signal read during its execution and re-runs whenever any of them changes. If the effect also *writes* to one of the signals it reads, that write re-triggers the same effect, which reads the (now-changed) signal again, potentially writing again — an infinite (or at least very wasteful)

cycle. Angular added `NG0600` specifically to catch this class of bug early, since it's a very common mistake when developers try to use `effect()` as a general-purpose reactive "watcher" for two-way state.

The correct pattern separates **pure derivation** (which belongs in `computed()`, not `effect()`) from **actual side effects** (touching the DOM, an external library) which belong in `effect()` and should never feed back into their own dependencies:

```
export class ZoomableChartComponent {
  // 1. Raw signal – the single source of truth, set only by user gesture handlers.
  private readonly rawZoom = signal(1);

  // 2. Pure derivation – clamping is a computed value, not a side effect. No
  //    infinite loop possible because computed() never writes to signals.
  readonly zoomLevel = computed(() => Math.min(4, Math.max(0.25, this.rawZoom())));

  constructor() {
    // 3. The only job of this effect is the actual side effect: telling the
    //    chart library to redraw. It reads zoomLevel but never writes to it
    //    or to rawZoom, so there's no cycle.
    effect(() => {
      const zoom = this.zoomLevel();
      this.chart?.zoomScale(zoom); // imperative call into chart library
    });
  }

  onWheel(delta: number) {
    // writes happen only from the actual user-input handler – never inside
    // the effect that reacts to the value.
    this.rawZoom.update(z => z + delta * 0.01);
  }
}
```

If you truly need an effect to conditionally write to a *different* signal it doesn't read (a legitimate pattern, e.g., syncing to a store), that's fine and doesn't loop. The danger is specifically reading-and-writing-the-same-signal (or a signal derived from it) inside one `effect()`.

Interviewer intent: Tests solid understanding of the signals mental model — specifically the difference between `computed()` (pure, no writes) and `effect()` (impure, side-effect-only, must not create feedback cycles) — a very frequent real-world gotcha in signal-based Angular code.

Scenario 14: Lazy-loading a heavy charting library only when the chart actually enters the viewport

Situation: The app bundles ECharts (a large library) directly into the main bundle, even though only 1 of 8 dashboard tabs actually shows a chart, and many users never click that tab. Bundle analysis shows ECharts contributes ~300KB gzipped to the initial load.

Question: How do you defer loading the charting library's code until it's actually needed?

Answer: Two independent lazy-loading concerns should be addressed:

1. **Route-level code splitting** — if the chart lives on its own route, standalone component lazy-loading

(`LoadComponent`) already keeps it and its imports out of the main bundle; this may already be solved if routing is set up correctly.

2. **Component-level deferred loading within a page that DOES eagerly load** — this is where Angular's `@defer` block shines. `@defer (on viewport)` doesn't just delay the component's rendering, it also **code-splits the deferred component (and its transitive imports, including the charting library) into a separate lazy chunk automatically** — no manual dynamic `import()` needed for the component itself, though the component's *internal* use of the library should still dynamically `import()` the library if you want maximum control over exactly when the 300KB parses/executes.

```
// dashboard.component.html
@defer (on viewport; prefetch on idle) {
  <app-echarts-panel [data]="salesData()" />
} @placeholder (minimum 200ms) {
  <div class="chart-skeleton">Chart loads when scrolled into view...</div>
} @loading (minimum 300ms; after 100ms) {
  <app-spinner />
} @error {
  <p>Could not load the chart.</p>
}
```

```
// echarts-panel.component.ts – the heavy import lives inside this deferred component.
import { Component, ElementRef, ViewChild, Input, AfterNextRender, DestroyRef, Inject } from
'@angular/core';

@Component({
  selector: 'app-echarts-panel',
  standalone: true,
  template: `<div #chartRef style="width:100%;height:400px;"></div>`,
})
export class EchartsPanelComponent {
  private readonly chartRef = ViewChild.required<ElementRef<HTMLDivElement>>>('chartRef');
  private readonly destroyRef = Inject(DestroyRef);
  data = Input.required<number[]>();

  constructor() {
    afterNextRender(async () => {
      const echarts = await import('echarts'); // heavy library, only parsed here
      const instance = echarts.init(this.chartRef().nativeElement);
      instance.setOption({ series: [{ type: 'line', data: this.data() }] });
      this.destroyRef.onDestroy(() => instance.dispose());
    });
  }
}
```

This gives three layers of laziness: (1) the deferred block doesn't even render the component until scrolled into view, (2) Angular's build tooling code-splits `EchartsPanelComponent` into its own chunk because it's referenced only from a `@defer` block, and (3) the `import('echarts')` inside that component further ensures the library's code doesn't execute until the component actually initializes — useful if the component itself has other lightweight logic you want available sooner.

Tradeoffs: `prefetch on idle` trades a bit of unnecessary network usage (for users who never scroll to the

chart) for near-instant rendering once they do scroll — a good default for common/likely-to-be-viewed content; use `on interaction` instead for genuinely optional/rare content.

Interviewer intent: Tests knowledge of `@defer`'s automatic code-splitting behavior (not just its rendering-delay behavior) and the ability to combine it with dynamic imports for maximum bundle discipline.

Scenario 15: Testing a chart component without actually rendering a real chart in unit tests

Situation: A team's unit tests for a chart component are slow and flaky in CI — they run inside `TestBed`, which triggers real Chart.js instantiation against a jsdom `<canvas>` that doesn't fully support the 2D rendering context, causing sporadic errors unrelated to the actual logic being tested (input handling, event emission).

Question: How should you structure the component and its tests so business logic is testable without depending on real canvas rendering?

Answer: The fix is a mix of testing strategy and a small refactor to make the chart library injectable/mockable:

1. **Wrap the third-party library behind an injectable adapter/service** (`ChartRenderersService` or similar) with a thin interface (`createChart`, `updateChart`, `destroyChart`). Production code injects the real Chart.js-backed implementation; tests provide a fake.
2. This makes the component's own logic (computing derived data, responding to inputs, emitting events on chart interaction) unit-testable in complete isolation from canvas/WebGL concerns, which are inherently hard to emulate faithfully in jsdom.
3. Reserve a handful of true integration/E2E tests (Playwright/Cypress against a real browser) to verify the actual rendering pipeline end-to-end — that's the appropriate layer for "does Chart.js actually draw pixels correctly," not unit tests.

```
// chart-renderer.service.ts – the seam that makes chart logic testable.
import { Injectable } from '@angular/core';

export interface ChartHandle {
  updateData(data: number[]): void;
  destroy(): void;
}

@Injectable({ providedIn: 'root' })
export class ChartRenderersService {
  create(canvas: HTMLCanvasElement, initialData: number[]): ChartHandle {
    // Real implementation constructs a Chart.js instance here.
    const chart = new Chart(canvas, { type: 'line', data: { datasets: [{ data: initialData
    }] } });
    return {
      updateData: (data) => { chart.data.datasets[0].data = data; chart.update(); },
      destroy: () => chart.destroy(),
    };
  }
}
```

```
// chart.component.spec.ts
describe('ChartWrapperComponent', () => {
```

```

it('emits pointSelected when a data point is clicked', () => {
  const fakeHandle: ChartHandle = { updatedData: jasmine.createSpy(), destroy:
jasmine.createSpy() };
  const fakeRenderer = { create: () => fakeHandle } as Partial<ChartRenderersService>;

  TestBed.configureTestingModule({
    imports: [ChartWrapperComponent],
    providers: [{ provide: ChartRenderersService, useValue: fakeRenderer }],
  });

  const fixture = TestBed.createComponent(ChartWrapperComponent);
  const component = fixture.componentInstance;
  spyOn(component.pointSelected, 'emit');

  component.simulateClickAt(2); // internal helper that calls the same code path a real
click handler would
  expect(component.pointSelected.emit).toHaveBeenCalled();
});
});

```

Tradeoffs: The adapter layer adds a small amount of indirection for a fairly thin wrapper, but it pays for itself immediately in CI stability and test speed — you get fast, deterministic unit tests for 95% of the component's logic, and only need a handful of slow, real-browser tests for actual visual/rendering correctness.

Interviewer intent: Tests whether the candidate designs for testability up front (dependency inversion around the un-mockable rendering primitive) rather than fighting jsdom's canvas limitations after the fact.

Scenario 16: Chart tooltips render behind other elements / get clipped by `overflow: hidden`

Situation: A chart lives inside a card component with `overflow: hidden` (to clip rounded corners). Tooltips that Chart.js renders as absolutely-positioned HTML elements near the cursor get visually clipped whenever the cursor is near the card's edge.

Question: How do you fix tooltip clipping without removing the `overflow: hidden` the design system relies on?

Answer: The classic CSS stacking-context problem: an `overflow: hidden` ancestor clips absolutely-positioned descendants regardless of `z-index`. There's no way to have both "clip the chart's rounded corners" and "let the tooltip overflow the card" if the tooltip is a literal DOM descendant of that clipped container — you must move the tooltip element **out of the clipped subtree** using a portal, while keeping it visually anchored to the cursor/data point.

Approach using Angular CDK Overlay (or manually appending to `document.body`):

1. Configure the chart library to render tooltips via a custom renderer/callback instead of its default absolutely-positioned-inside-container HTML tooltip.
2. In that callback, compute the tooltip's page-level `(x, y)` position and use Angular CDK's `overlay / overlayRef` (which attaches to the `<body>`, entirely outside any clipped ancestor) to render the tooltip content as an Angular component, positioned imperatively.
3. Tear the overlay down on `mouseleave` / chart destroy.

```

import { Component, ElementRef, ViewChild, inject, DestroyRef, afterNextRender } from
'@angular/core';
import { Overlay, OverlayRef } from '@angular/cdk/overlay';
import { ComponentPortal } from '@angular/cdk/portal';
import { TooltipContentComponent } from './tooltip-content.component';

@Component({
  selector: 'app-chart-with-portal-tooltip',
  standalone: true,
  template: `<canvas #canvasRef></canvas>`,
})
export class ChartWithPortalTooltipComponent {
  private readonly canvasRef = ViewChild.required<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly overlay = inject(Overlay);
  private readonly destroyRef = inject(DestroyRef);
  private overlayRef?: OverlayRef;

  constructor() {
    afterNextRender(() => {
      const chart = new Chart(this.canvasRef().nativeElement, {
        type: 'line',
        data: { datasets: [] },
        options: {
          plugins: {
            tooltip: {
              enabled: false, // disable Chart.js's built-in clipped tooltip
              external: (context) => this.renderExternalTooltip(context),
            },
          },
        },
      });
      this.destroyRef.onDestroy(() => { chart.destroy(); this.overlayRef?.dispose(); });
    });
  }

  private renderExternalTooltip(context: any) {
    const { tooltip } = context;
    if (tooltip.opacity === 0) { this.overlayRef?.dispose(); this.overlayRef = undefined;
    return; }

    const canvasRect = this.canvasRef().nativeElement.getBoundingClientRect();
    const positionStrategy = this.overlay.position()
      .global()
      .left(`${canvasRect.left + tooltip.caretX}px`)
      .top(`${canvasRect.top + tooltip.caretY}px`);

    if (!this.overlayRef) {
      this.overlayRef = this.overlay.create({ positionStrategy, scrollStrategy:
this.overlay.scrollStrategies.reposition() });
      this.overlayRef.attach(new ComponentPortal(TooltipContentComponent));
    } else {
      this.overlayRef.updatePositionStrategy(positionStrategy);
    }
  }
}

```

```
}  
}  
}
```

Tradeoffs: Using CDK Overlay for tooltips is more code than the library's built-in tooltip, but it's the only robust fix once a clipped ancestor is a hard design requirement; the alternative — removing `overflow: hidden` — is usually rejected by design for the rounded-corner requirement, so the portal approach becomes necessary rather than optional.

Interviewer intent: Tests whether the candidate understands CSS stacking-context/clipping fundamentals well enough to recognize that no z-index value fixes clipping by an `overflow: hidden` ancestor, and knows Angular CDK Overlay as the standard escape hatch.

Scenario 17: Switching chart color themes (light/dark mode) without recreating the whole chart

Situation: The app supports a dark-mode toggle. Currently, toggling dark mode destroys and recreates every chart on the page to apply new colors, causing a visible flash and losing any zoom/pan state the user had set.

Question: How do you re-theme charts smoothly when the app's color scheme changes?

Answer: Model the color theme itself as a signal (or derive it from a `prefers-color-scheme` media query / app-level theme service), and in the chart's `effect()`, update only the color-related chart options in place — never destroy/recreate the whole instance just for a palette change, since the chart's data, zoom state, and scales don't need to be rebuilt.

```
import { Injectable, signal, effect } from '@angular/core';  
  
@Injectable({ providedIn: 'root' })  
export class ThemeService {  
  readonly isDark = signal(window.matchMedia('(prefers-color-scheme: dark)').matches);  
  toggle() { this.isDark.update(v => !v); }  
}
```

```
export class ThemedChartComponent {  
  private readonly theme = inject(ThemeService);  
  private chart?: Chart;  
  
  private readonly palette = computed(() => this.theme.isDark()  
    ? { grid: '#333', text: '#eee', line: '#8ab4f8' }  
    : { grid: '#e0e0e0', text: '#111', line: '#1a73e8' }  
  );  
  
  constructor() {  
    effect(() => {  
      const colors = this.palette();  
      if (!this.chart) return;  
      // Update only color-related options; leave data/zoom/scale-range untouched.  
      this.chart.options.scales!['x']!.grid!.color = colors.grid;  
      this.chart.options.scales!['y']!.grid!.color = colors.grid;  
      this.chart.options.plugins!.legend!.labels!.color = colors.text;  
      this.chart.data.datasets[0].borderColor = colors.line;  
    });  
  }  
}
```

```
    this.chart.update('none'); // no animation flash for a theme swap
  });
}
```

Tradeoffs: This requires knowing which specific chart-option paths correspond to "color" versus "structural" configuration for whichever library you use — slightly more library-specific knowledge than a blunt destroy/recreate, but it eliminates flicker and preserves interaction state (zoom, pan, selected legend items), which matters a lot for a good dark-mode-toggle UX.

Interviewer intent: Tests the instinct to prefer targeted, minimal updates over "nuke and rebuild" whenever only a subset of configuration actually changed — a recurring theme across chart integration questions.

Scenario 18: A chart showing stale data because of a race between async data fetch and component destruction

Situation: A chart component fetches its dataset from an HTTP endpoint in `ngOnInit`/constructor via `inject(HttpClient).get(...)`. Users who quickly navigate away from the chart's page and back sometimes see the *previous* dataset flash briefly before the new one loads, and occasionally see a console error about setting state on a destroyed component.

Question: What's happening, and how do you fix it using modern Angular idioms?

Answer: This is a classic async-race-plus-missing-cancellation bug: the HTTP request from the *first* mount is still in flight when the component is destroyed (route navigated away) and then re-created (navigated back); when the first request finally resolves, its `.subscribe()` callback fires against a component instance that isn't the current one — or worse, closures reference component state that should've been discarded, causing stale-data flashes or errors if the callback tries to touch destroyed view resources.

Fix: **cancel in-flight requests on destroy** using `takeUntilDestroyed()` (which uses `DestroyRef` internally and is the idiomatic modern replacement for manual `Subject`-based teardown), and derive chart data via signals fed by `toSignal()` so there's no manual subscription bookkeeping at all.

```
import { Component, inject, computed } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { toSignal, takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { of } from 'rxjs';
import { catchError } from 'rxjs/operators';

@Component({
  selector: 'app-async-chart',
  standalone: true,
  template: `<app-chart [data]="chartData()" />`,
})
export class AsyncChartComponent {
  private readonly http = inject(HttpClient);

  // toSignal automatically unsubscribes when the component is destroyed —
  // no manual takeUntilDestroyed needed for this specific call, but it's
  // shown below for the case where you need side effects mid-stream too.
  private readonly rawData = toSignal(
    this.http.get<number[]>('/api/chart-data').pipe(
```

```

        catchError(() => of([] as number[]))
    ),
    { initialValue: [] as number[] }
);

chartData = computed(() => this.rawData());
}

```

```

// If you need imperative subscribe() instead of toSignal (e.g., to also
// trigger a non-signal side effect), use takeUntilDestroyed explicitly:
export class AsyncChartComponentImperative {
    private readonly http = inject(HttpClient);
    private readonly destroyRef = inject(DestroyRef);
    data = signal<number[]>([]);

    constructor() {
        this.http.get<number[]>('/api/chart-data')
            .pipe(takeUntilDestroyed(this.destroyRef))
            .subscribe(result => this.data.set(result));
    }
}

```

Because `toSignal` / `takeUntilDestroyed` tie the subscription's lifetime directly to the component's `DestroyRef`, a stale response arriving after destruction is simply never delivered — it can't touch a signal or trigger an effect on a component that no longer exists, which eliminates both the stale-flash symptom and the destroyed-component console error.

Interviewer intent: Tests fluency with `rxjs-interop` (`toSignal`, `takeUntilDestroyed`) as the modern replacement for manual `ngOnDestroy` + `Subject<void>` teardown patterns, and understanding of why uncanceled async work causes race conditions across re-mounts.

Scenario 19: Combining `@defer` with a loading skeleton that matches the final chart's dimensions to avoid layout shift

Situation: After introducing `@defer (on viewport)` for chart panels (Scenario 14), Core Web Vitals monitoring shows a regression in Cumulative Layout Shift (CLS) — when the deferred chart finally loads, the page content around it jumps because the placeholder was a different height than the real chart.

Question: How do you prevent the layout shift while still deferring the actual chart library's loading?

Answer: CLS regressions from `@defer` almost always come from the `@placeholder` block not reserving the same box size as the eventually-rendered content. The fix has nothing to do with the charting library itself — it's disciplined CSS sizing across all three defer states (`@placeholder`, `@loading`, and the real content):

1. Give the **outer container** (not the placeholder or the chart individually) a fixed or aspect-ratio-based height that's identical regardless of which of the three states is currently showing.
2. Make the placeholder, loading indicator, and the real chart's host element all inherit/fill that same fixed-size container (`height: 100%`), so swapping between them never changes the container's box size — the container's size is set once, structurally, and none of the conditionally-rendered children can affect it.

3. If chart heights genuinely vary by data (e.g., a dynamic-height bar chart), use `aspect-ratio` or a server-known/pre-computed height hint passed down as a signal/input, rather than letting the chart size itself only after its JS runs.

```
<!-- The fixed-size wrapper is OUTSIDE the @defer block, so its size never
      changes across placeholder / loading / real-content swaps. -->
<div class="chart-slot">
  @defer (on viewport; prefetch on idle) {
    <app-echarts-panel [data]="salesData()" />
  } @placeholder {
    <div class="chart-slot__fill chart-skeleton"></div>
  } @loading (minimum 300ms; after 100ms) {
    <div class="chart-slot__fill"><app-spinner /></div>
  } @error {
    <div class="chart-slot__fill chart-error">Failed to load chart</div>
  }
</div>
```

```
.chart-slot {
  /* Reserved size is set once, structurally – never affected by which
     conditional branch is currently rendered inside it. */
  height: 400px;
  width: 100%;
  position: relative;
}
.chart-slot__fill {
  position: absolute;
  inset: 0;
}
.chart-skeleton {
  background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%);
  background-size: 200% 100%;
  animation: shimmer 1.2s infinite;
}
@keyframes shimmer { from { background-position: 200% 0; } to { background-position: -200% 0; } }
```

Tradeoffs: A fixed pixel height is simplest but breaks down for genuinely variable-height content; `aspect-ratio` handles responsive width changes gracefully but still needs *some* known ratio ahead of time — if truly unknown until data loads, consider server-computing an estimated height (e.g., from row count) and passing it down so the reserved space is a close approximation rather than a guess.

Interviewer intent: Tests whether the candidate connects a Core Web Vitals metric (CLS) back to a concrete `@defer`/CSS-sizing root cause, rather than treating `@defer` as a black box that "just improves performance" with no downside to manage.

Scenario 20: Chart library's zoom/pan plugin conflicts with the page's own scroll behavior

Situation: A chart uses a zoom/pan plugin (e.g., `chartjs-plugin-zoom`) so users can scroll-wheel to zoom into a time-series chart. Unfortunately, scrolling the mouse wheel while hovering over the chart also scrolls the entire page, so users end up scrolling the page and zooming the chart simultaneously — a confusing, janky experience.

Question: How do you cleanly separate "the user wants to zoom the chart" from "the user wants to scroll the page" for the same wheel gesture?

Answer: This is fundamentally an event-propagation and default-action problem: a `wheel` event over the canvas bubbles up to the page and triggers native scrolling *unless* something calls `event.preventDefault()`. The chart plugin usually handles zoom but may not aggressively prevent the default scroll in all cases (especially with passive listener defaults in modern browsers, which by spec make `preventDefault()` a no-op unless the listener was registered as non-passive).

Fix, layered:

1. Ensure the zoom plugin's wheel listener (or your own wrapper listener) is registered with `{ passive: false }` explicitly — modern browsers default `wheel / touchstart` listeners to passive for scroll-performance reasons, which silently makes `preventDefault()` do nothing unless you opt out.
2. Only intercept the wheel event (and thus block page scroll) while the cursor is actually over the chart **and** typically require a modifier key (e.g., Ctrl/Cmd+scroll to zoom, plain scroll to pass through to the page) — this is the same convention Google Maps and most mapping/charting UIs use, and it avoids trapping users who are just trying to scroll past the chart.
3. Clean up the listener via `DestroyRef` like any other manually-added DOM listener, since Angular's own `(wheel)` template binding doesn't give you control over `passive`.

```
import { Component, ElementRef, ViewChild, inject, DestroyRef, afterNextRender } from
'@angular/core';
import { Chart } from 'chart.js/auto';
import zoomPlugin from 'chartjs-plugin-zoom';

Chart.register(zoomPlugin);

@Component({
  selector: 'app-zoomable-chart',
  standalone: true,
  template: `<canvas #canvasRef></canvas><p class="hint">Ctrl/Cmd + scroll to zoom</p>`,
})
export class ZoomableChartComponent {
  private readonly canvasRef = ViewChild<ElementRef<HTMLCanvasElement>>
('canvasRef');
  private readonly destroyRef = inject(DestroyRef);

  constructor() {
    afterNextRender(() => {
      const canvas = this.canvasRef().nativeElement;
      const chart = new Chart(canvas, {
        type: 'line',
        data: { datasets: [] },
```

```

    options: {
      plugins: {
        zoom: {
          zoom: { wheel: { enabled: true, modifierKey: 'ctrl' }, mode: 'x' },
          pan: { enabled: true, mode: 'x' },
        },
      },
    },
  },
});

// Explicitly non-passive so preventDefault() actually stops page
// scroll – but ONLY when the modifier key is held, so plain scrolling
// still passes through to the page for everyone else.
const wheelHandler = (evt: WheelEvent) => {
  if (evt.ctrlKey || evt.metaKey) evt.preventDefault();
};
canvas.addEventListener('wheel', wheelHandler, { passive: false });

this.destroyRef.onDestroy(() => {
  canvas.removeEventListener('wheel', wheelHandler);
  chart.destroy();
});
});
}
}

```

Tradeoffs: Requiring a modifier key adds a tiny discoverability cost (hence the visible hint text), but it's the industry-standard resolution to this exact conflict and is far preferable to either (a) always hijacking scroll on hover (breaks page scrolling for anyone passing over the chart) or (b) never preventing default (chart zoom becomes nearly unusable because every wheel tick also scrolls the page out from under the cursor).

Interviewer intent: Tests knowledge of passive event listener semantics (a common footgun since Chrome's passive-by-default change) and the UX judgment to solve gesture conflicts with an established convention rather than a one-off hack.

Quick Revision Cheat Sheet

- **One-way data flow into imperative libraries:** feed chart libraries via `input()` signals and react with `effect()`; never let the chart library write back into Angular state directly — keep Angular as the source of truth.
- **Create in `afterNextRender`, destroy via `DestroyRef`:** this pairing is the modern replacement for `ngOnInit / ngOnDestroy` for anything touching the DOM/canvas, and `afterNextRender` is inherently SSR-safe.
- **Immutability matters even with signals:** mutating arrays/objects in place can bypass memoization (`computed()`) and confuse chart libraries that cache references — always produce new references for changed data.
- **Update in place before you destroy/recreate:** flicker, lost zoom/pan state, and wasted CPU almost always trace back to unnecessarily tearing down and rebuilding a chart instance when a targeted `chart.update()` would have sufficed (theming, data changes, resizing).

- **Isolate DOM ownership:** never let Angular structural directives (`*ngFor`, `@for`) and an imperative library (D3, Three.js) manage children of the same container — give the library one exclusive host element.
- **Run heavy/frequent chart work outside the zone:** `NgZone.runOutsideAngular()` (or plain `zoneless` + signals) prevents chart-library-internal work (animations, hover, streaming redraws) from triggering unrelated Angular change detection.
- **Batch and window streaming data:** for high-frequency updates, buffer with `bufferTime`, cap the dataset with a sliding window, and disable animation (`update('none')`) — never redraw once per incoming message.
- `ResizeObserver` **beats** `window:resize` for any chart inside a flexible/resizable container; observe the container, not the canvas itself, to avoid feedback loops.
- **SSR crashes come from module-load-time global access**, not just runtime access — use dynamic `import()` plus `afterNextRender` / `isPlatformBrowser` guards so browser-only chart libraries never execute (or even parse) on the server.
- **Canvas is invisible to assistive tech** — pair every chart with `role="img"` + a meaningful `aria-label` summary, and a visually-hidden (or toggleable) data table for screen reader and keyboard users.
- `effect()` **must not read and write the same signal** — use `computed()` for pure derivation (like clamping) and reserve `effect()` purely for side effects like calling into the chart library, or you'll hit `NG0600` /infinite loops.
- `@defer` **code-splits automatically** — use viewport/interaction triggers to keep heavyweight charting libraries out of the initial bundle, and pair with fixed-size wrapper containers to avoid CLS regressions.
- **Cancel in-flight async chart-data requests** with `takeUntilDestroyed()` / `toSignal()` so fast navigation away-and-back can't deliver stale data to a re-created component.
- **Portal tooltips out of clipped ancestors** with Angular CDK Overlay when a design system's `overflow: hidden` conflicts with a chart library's absolutely-positioned tooltip.
- **Passive event listeners silently defeat** `preventDefault()` — register wheel/touch handlers as `{ passive: false }` explicitly whenever a chart's zoom/pan gesture needs to override the page's native scroll behavior.
- **Test business logic, not canvas rendering:** wrap the charting library behind a small injectable adapter so unit tests can substitute a fake renderer, and reserve real-browser E2E tests for actual pixel-level rendering verification.

Created By - Durgesh Singh